ESCUELA SUPERIOR POLITÉCNICA AGROPECUARIA DE MANABÍ MANUEL FÉLIX LÓPEZ

CARRERA:

DESARROLLO DE SOFTWARE EN LÍNEA

ASIGNATURA:

ESTRUCTURAS DE DATOS Y ALGORITMOS

TEMA:

GRUPO 3: LISTAS DOBLEMENTE ENLAZADAS Y CIRCULARES (LISTA DE REPRODUCCIÓN MULTIMEDIA CON BUCLE CONTINUO - PLAYLIST).

ESTUDIANTE:

MADELINE RAMÍREZ TUMBACO

DOCENTE:

LUIGGI ALEXANDER JALCA SALTOS

PARALELO:

A

27 DE OCTUBRE DE 2026

REPOSITORIO:

https://github.com/Rmadeline/EDA-2N-ESPAM-RM

Objetivo Pedagógico

Justificación técnica del uso de Java para implementar manualmente estructuras lineales (Arreglos, Listas, Pilas, Colas) y resolver el caso de la Playlist.

Sección 1- Análisis del Problema

Caso: Lista de Reproducción Multimedia con Bucle Continuo (Playlist).

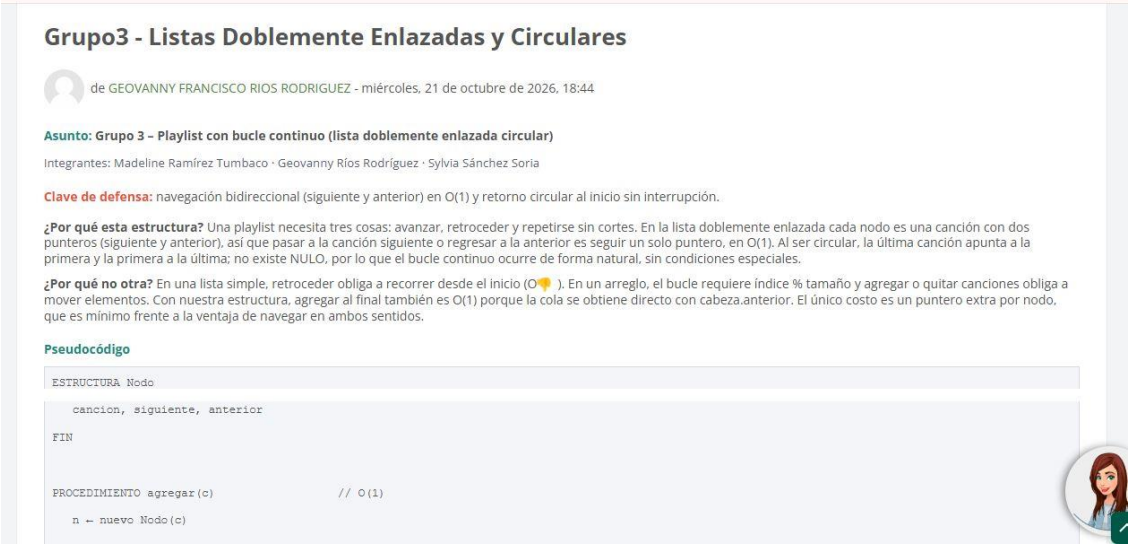
Justificación Técnica: Una playlist requiere recorrer canciones hacia adelante y hacia atrás, así como repetirse infinitamente sin interrupciones. La **Lista Doblemente Enlazada Circular** resuelve esto de forma óptima porque:

- Cada nodo (canción) tiene dos punteros (siguiente y anterior), permitiendo navegación bidireccional en tiempo constante O(1).
- Al ser circular, el último nodo apunta al primero (y el primero al último), logrando un bucle continuo sin condiciones especiales ni valores nulos (null).
- La inserción o eliminación de canciones al final o al inicio se realiza en O(1) mediante la referencia al nodo cabeza (cabeza.anterior).

Mapeo de requerimientos para la playlist con navegación bidireccional en O(1)

- **Situación (S):** Desarrollo de un reproductor multimedia digital que gestiona una lista de canciones para reproducción secuencial e interactiva.
- **Complicación (C):** Usar un arreglo estático impide agregar canciones dinámicamente y exige mover elementos en memoria; usar una lista simple impide retroceder a la canción anterior en O(1) y requiere validaciones manuales para reiniciar la lista al terminar.
- **Pregunta (Q):** ¿Cómo estructurar los datos para garantizar navegación bidireccional inmediata en O(1) y reproducción circular infinita en memoria Heap con mínimo sobrecosto?
- **Respuesta (A):** Implementar una **Lista Doblemente Enlazada Circular** en memoria dinámica Heap, asignando dos punteros por nodo para lograr navegación transparente en ambas direcciones y bucle automático.

Justificación del caso usando la técnica SCQA (Situación, Complicación, Pregunta, Respuesta).



Grupo3 - Listas Doblemente Enlazadas y Circulares

de GEOVANNY FRANCISCO RIOS RODRIGUEZ - miércoles, 21 de octubre de 2026, 18:44

Asunto: Grupo 3 – Playlist con bucle continuo (lista doblemente enlazada circular)

Integrantes: Madeline Ramírez Tumbaco · Geovanny Ríos Rodríguez · Sylvia Sánchez Soria

Clave de defensa: navegación bidireccional (siguiente y anterior) en O(1) y retorno circular al inicio sin interrupción.

¿Por qué esta estructura? Una playlist necesita tres cosas: avanzar, retroceder y repetirse sin cortes. En la lista doblemente enlazada cada nodo es una canción con dos punteros (siguiente y anterior), así que pasar a la canción siguiente o regresar a la anterior es seguir un solo puntero, en O(1). Al ser circular, la última canción apunta a la primera y la primera a la última; no existe NULO, por lo que el bucle continuo ocurre de forma natural, sin condiciones especiales.

¿Por qué no otra? En una lista simple, retroceder obliga a recorrer desde el inicio (O👎). En un arreglo, el bucle requiere índice % tamaño y agregar o quitar canciones obliga a mover elementos. Con nuestra estructura, agregar al final también es O(1) porque la cola se obtiene directo con cabeza.anterior. El único costo es un puntero extra por nodo, que es mínimo frente a la ventaja de navegar en ambos sentidos.

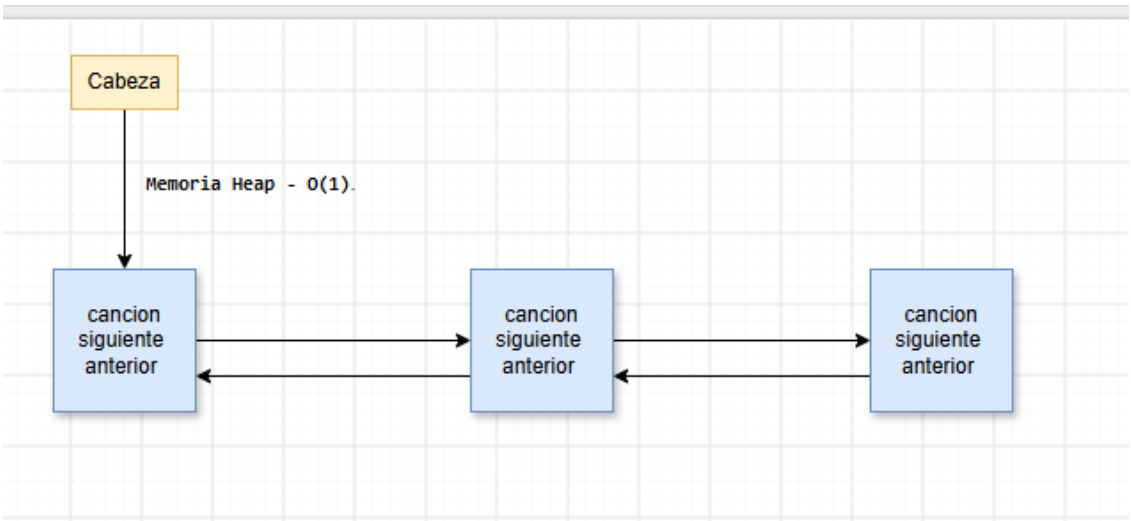
Pseudocódigo

```
ESTRUCTURA Nodo
    cancion, siguiente, anterior
FIN


PROCEDIMIENTO agregar(c)                    // O(1)
    n ← nuevo Nodo(c)
```

Captura de la participación y réplica técnica en el foro de Moodle (Clase 06).

Sección 02: Diagramas



Modelado en Draw.io de la lista circular en memoria Heap con punteros siguiente, anterior y cabeza.

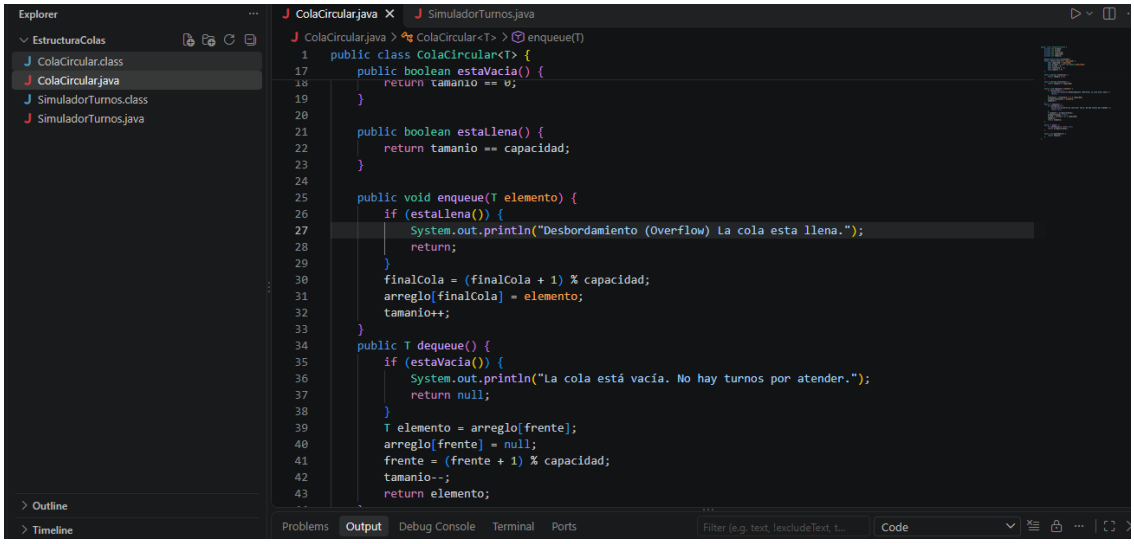
Sección 03: Código Java

```
ESTRUCTURA Nodo
    cancion, siguiente, anterior
FIN

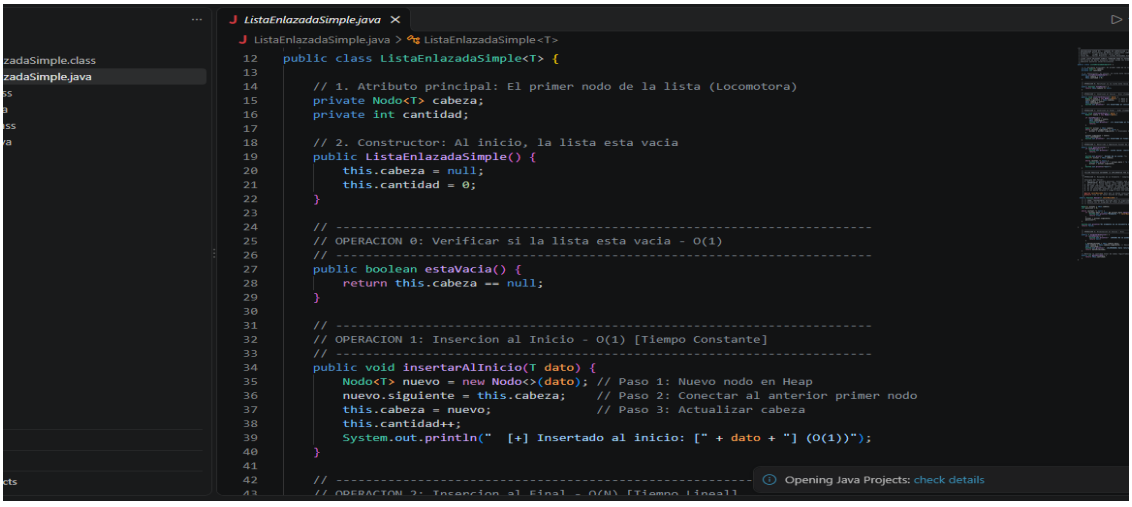
PROCEDIMIENTO agregar(c)                    // O(1)
    n ← nuevo Nodo(c)
    SI cabeza = NULO ENTONCES
        cabeza ← n;   n.sig ← n;   n.ant ← n
        actual ← n
    SINO
        cola ← cabeza.ant                    // sin recorrer
        cola.sig ← n;     n.ant ← cola
        n.sig ← cabeza;   cabeza.ant ← n
    FIN SI
FIN

PROCEDIMIENTO siguiente()
    actual ← actual.sig                      // O(1)
PROCEDIMIENTO anterior()
    actual ← actual.ant                      // O(1)
```

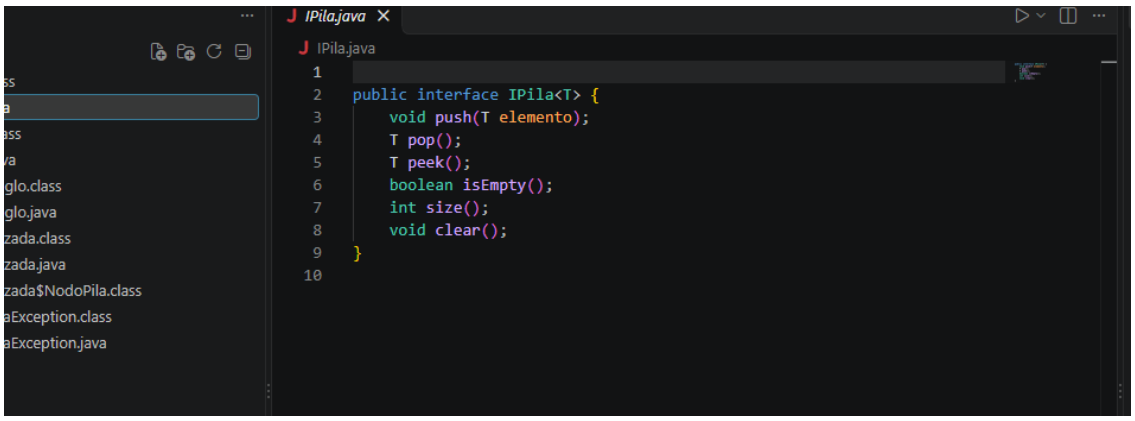
Implementación de la lista doble circular para la playlist



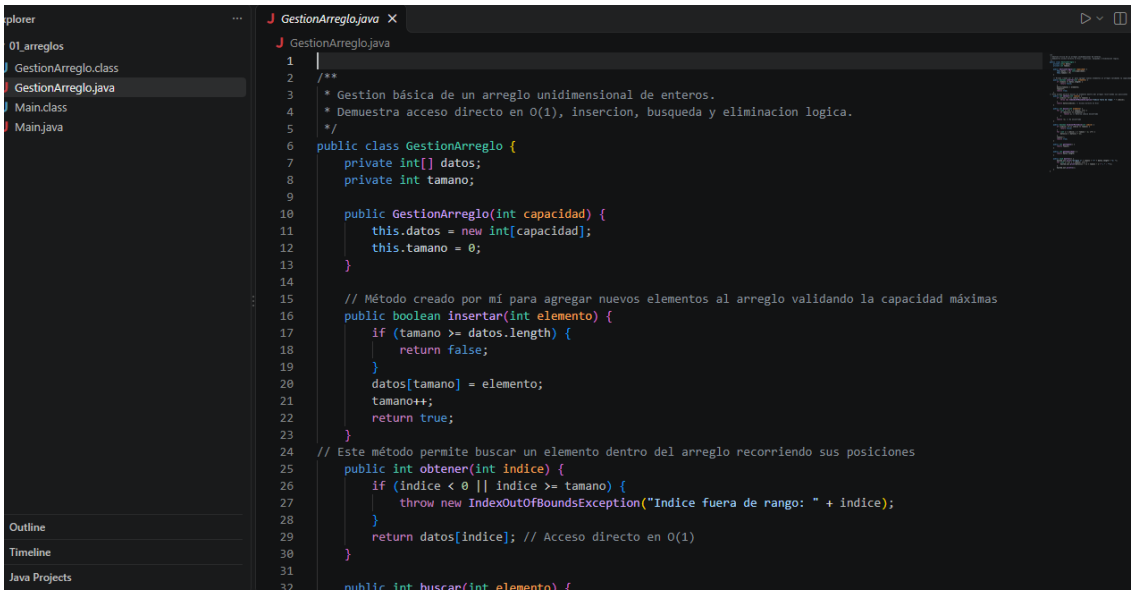
Cola FIFO con fórmula modular (rear + 1) % capacidad para evitar falso desbordamiento.



Creación de nodos con referencias en memoria Heap.



Métodos push, pop, peek bajo principio LIFO y control de desbordamiento.



Estructura contigua con acceso directo por índice en O(1).



Consola de ejecución integrada que demuestra el funcionamiento de todas las estructuras.

Conclusión técnica

Análisis de Complejidad Temporal Big-O: Se comprobó que las estructuras contiguas como los arreglos permiten un acceso directo a los elementos en tiempo constante O(1) mediante índices, mientras que las estructuras enlazadas basadas en nodos (como la lista doble circular de la Playlist) optimizan las inserciones y la navegación bidireccional continua sin requerir memoria contigua fija.

Aplicación al Caso Real: La implementación manual en Java de la lista de reproducción multimedia demostró ser la solución idónea para garantizar un bucle continúo fluido, validando el dominio algorítmico y estructural de la Unidad 1.

Gestión de Memoria (Stack vs Heap): Se evidenció la diferencia práctica entre almacenar variables estáticas en el Stack y asignar nodos dinámicos en el Heap, utilizando punteros enlazados y aritmética modular en las colas circulares para evitar el falso desbordamiento.